

Task-Aware Prompt Calibration for Generating Synthetic Novice Python Code

Aidil Muslim
amus790@aucklanduni.ac.nz
University of Auckland
Auckland, New Zealand

ABSTRACT

Large language models offer a potential source of synthetic code for evaluating educational analysis and feedback systems. However, synthetic code may not reproduce the stylistic and structural defects found in authentic novice submissions. This study proposes a task-aware, iterative prompt-calibration approach for generating functionally correct Python code with realistic novice-style defects. Authentic CS1 submissions will be analysed to classify defects as task-independent or task-dependent. The classification will develop target defect profiles that guide prompt construction and refinement. A lightweight prototype will compare several prompting methods through functional correctness tests, profile similarity comparison, and category coverage. This research intends to build a reusable method for producing synthetic novice code that is closer to authentic student code.

CCS Concepts

• Social and professional topics~ Professional topics~ Computing education~ Computing education programs~ Computer science education~ CS1

Keywords

novice programming; code style defects; synthetic data; AI code generation; prompt engineering

1. Introduction

Introductory programming courses increasingly utilise automated assessment, static analysis, and large language models (LLMs) to support the learning activities. Although functional tests determine whether a program produces expected outputs, they do not depict the readability, maintainability, and structure of the submissions. Effenberger and Pelánek (2022) analysed 114,000 correct Python solutions to 161 introductory problems and identified 32 code-quality defects, finding that most correct solutions contained defects and that their prevalence varied across topics such as expressions, loops, and lists (Effenberger & Pelánek, 2022). Later research constructed a broader catalogue of introductory-programming defects that includes defect types produced in introductory programming courses, such as duplication, unsuitable naming, unused variables, formatting problems, and inappropriate control structures (Rechackova et al., 2024, 2025). At the same time, Artificial Intelligence (AI) code generators can assist novice learners, while educational tools such as PythonTA demonstrate the value of code analysis designed specifically for beginners

(Kazemitabaar et al., 2023; Liu et al., 2024). These developments create a need for realistic code samples that represent not merely functional correctness but also the stylistic and structural tendencies found in authentic computer science first-year (CS1) submissions.

Authentic student submissions are essential for educators and automated feedback systems developers to test detection, feedback rules, and educative intervention before deploying the programs in a course. In reality, student submissions are limited and restricted by the privacy regulation, organisational access, limited task coverage, or even the time needed for data acquisition. LLMs have the potential to be an alternative for providing the dataset, especially where genuine data is scarce or sensitive. However, the lack of stylistic realism is one of the problems in synthetic code that can affect the reproducibility of the defect patterns found in student works (Nadáš et al., 2025). If synthetic examples overproduce noticeable defects or introduce defects that are implausible for a particular task, the code evaluations may provide an inaccurate characteristic of system performance. The practical problem in this case is how to produce functionally correct synthetic code that reflects typical, realistic novice student defect tendencies across different programming tasks.

Research on introductory programming has established that typical code-quality defects occur in preliminary programming courses and problems. (Effenberger & Pelánek, 2022; Rechackova et al., 2025). Other studies have developed novice-oriented static-analysis and feedback systems. At the same time, research on LLM-based synthetic data has examined prompt-based generation and iterative refinement in text and code domains (Liu et al., 2024; Nadáš et al., 2025). Taken together, these research streams do not yet provide an integrated method for controlling the distribution of stylistic and structural defects in functionally correct synthetic CS1 code. An aggregate defect profile may combine tendencies that are relatively stable across tasks with defects whose occurrence depends on requirements such as branching, iteration, validation, or decomposition. Tasks with larger submission counts may also dominate it. The research gap this study seeks to address is the absence of a task-aware, adaptive prompting approach that distinguishes defect type tendencies according to task dependence and uses empirical target profiles using real CS1 student submissions to iteratively refine prompts when generated defect distributions deviate from the intended targets.

This study aims to develop and evaluate a task-aware, iterative prompt-calibration approach for generating functionally correct synthetic Python code with realistic novice-style defects. Guided by this objective, the study addresses the following research questions:

RQ1: How can code-quality defect types in authentic CS1 Python submissions be classified as task-independent and task-dependent based on their occurrence patterns across programming tasks?

Commented [AM1]: Support with evidence

RQ2: How can empirical profiles of task-independent and task-dependent defects be incorporated into a task-aware prompting approach for generating functionally correct synthetic novice code?

RQ3: To what extent does iterative, task-aware prompt refinement improve the alignment between the defect distributions of synthetic code and authentic CS1 student submissions compared with a non-adaptive prompting approach?

2. Methodology

The study will follow a design-science research approach centred on developing and evaluating a purposeful artefact for a relevant educational problem (Nunamaker et al., 1990). The research process will progress from establishing a conceptual framework and defining artefact requirements to developing a prototype and conducting a structured evaluation (Nunamaker et al., 1990). First, authentic student code will be analysed to identify recurring stylistic and structural defect patterns, with the reliability of the analysis procedures validated through manual review. Programming tasks will then be examined according to their underlying characteristics to distinguish general defect tendencies from those influenced by task context. These empirical findings will guide the design of the prompting artefact and its iterative refinement. Generated code will be assessed for both functional correctness and defect representation. Evaluation results will be used to compare prompting approaches, assess consistency, and inform further improvements to the artefact.

3. Research Artefacts

The research will produce two primary artefacts and one supporting instantiation. The first artefact is a two-layer defect-tendency model that distinguishes task-independent tendencies from task-dependent tendencies associated with selected CS1 task families. This model will provide the empirical foundation for the iterative prompt calibration method that incorporates both profile layers into prompt construction. The method will compare generated defect distributions with the empirical target profiles, identify defect underproduction, and adjust the relevant prompt components accordingly. Together, these artefacts represent a model and a method within the design-science classification of constructs, models, methods, and instantiations (Hevner et al., 2004). A light-weight generation-and-evaluation prototype will instantiate the model and method, enabling their implementation, comparison, and iterative assessment. The anticipated contribution is a reusable design approach for generating functionally correct novice-style code that more closely reflects the general and task-specific defect patterns observed in authentic novice programming students.

4. Potential Paper Structure

The paper will next review literature on novice code quality, educational code-analysis systems, synthetic code generation, and design-science research. It will then describe the dataset, task-family construction, defect measurement, artefact requirements, and evaluation design. Subsequent sections will present the two-layer model, calibration method, prototype, and comparative evaluation. The discussion will interpret the findings and contributions, followed by limitations, future research, and the conclusion.

References

- Effenberger, T., & Pelanek, R. (2022). Code Quality Defects across Introductory Programming Topics. *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education (SIGCSE 2022)*. <https://doi.org/10.1145/3478431.3499415>
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Q.*, 28(1), 75–105. <https://doi.org/10.2307/25148625>
- Kazemitabaar, M., Chow, J., Ma, C., Ericson, B., Weintrop, D., & Grossman, T. (2023). Studying the effect of AI Code Generators on Supporting Novice Learners in Introductory Programming. *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. <https://doi.org/10.1145/3544548.3580919>
- Liu, D., Calver, J., & Craig, M. (2024). A Static Analysis Tool in CS1: Student Usage and Perceptions of PythonTA. *Proceedings of the 26th Australasian Computing Education Conference, ACE '24*, 172–181. <https://doi.org/10.1145/3636243.3636262>
- Nadăș, M., Dioșan, L., & Tomescu, A. (2025). Synthetic Data Generation Using Large Language Models: Advances in Text and Code. *IEEE Access*, 13, 134615–134633. <https://doi.org/10.1109/ACCESS.2025.3589503>
- Nunamaker, J. F., Chen, M., & Purdin, T. D. M. (1990). Systems Development in Information Systems Research. *Journal of Management Information Systems*, 7(3), 89–106. <https://doi.org/10.1080/07421222.1990.11517898>
- Rechackova, A., Pelanek, R., & Effenberger, T. (2024). Catalog of Code Quality Defects in Introductory Programming. *Proceedings of the 2024 Innovation and Technology in Computer Science Education Conference (ITiCSE 2024)*. <https://doi.org/10.1145/3649217.3653638>
- Rechackova, A., Pelanek, R., & Effenberger, T. (2025). Code Quality Defects in Introductory Programming. *ACM Transactions on Computing Education*, 25(4). <https://doi.org/10.1145/3749996>